DPUSGETextureQuick

Renders a HMI with contents defined in the QML language into an off-screen image and provides the image contents as a texture for the visualization SGE.

1. Inputs:

(*dynamically* defined)	Inputs are dynamically defined using the *In* parameter list (see below).

2. Dynamic parameters:

Active	Does the HMI currently react to changes to its properties? By temporarily setting the value to 0, significant GPU re-sources can be saved if the associated object isn't cur-rently visible. Default: 1

3. Static parameters:

MainFile	Main QML file which defines the HMI structure. The file is searched within the project and installation folders. Default: *qml/main.qml*
WindowW, WindowH	Size of the output image, in pixels. Default: 1024 x 768
View	Name of the SGE view for which the video texture is updated. Additionally, it will be visible in all views that share textures with the specified SGE view. Default: *SGEView*
TextureName	Name of the created video texture, which must start with a dollar sign (*$*). Default: *$Quick*

4. Outputs:

(dynamically defined)	Outputs are dynamically defined using the *Out* parameter list (see below).

5. Functional principle:

Loads the specified QML file and creates the specified user interface. The user interface is rendered into an off-screen image. The contents of this image are made available to the visualization SGE as a dynamic texture.

The *TextureName* parameter defines a name under which the dynamic texture will be available in the SGE world. The name must start with a dollar sign (*$*). The dynamic texture can be used in any SGE object loaded after this DPU by using this texture name instead of specifying a texture file on the hard

disk. Alternatively, *DPUSGEVideoObject* can be used to create a simple overlay object which displays the dynamic texture.

As the user interface is rendered off-screen, direct interaction using keyboard, mouse and touch devices is not possible. However, it is possible to transfer data between the HMI and other SILAB DPUs by defining additional inputs and outputs (see below). This can be used to provide the value of input controls (such as hard-ware buttons in the simulator mockup) to the HMI and modify the status of the user interface accordingly.

6. Defining inputs and outputs:

The In and Out parameter list define which properties in the QML structure are "imported from" and "exported to" SILAB. The syntax is as follows:

To import properties from SILAB:

```
In = {
      element1.property1,
      element2.property2,
      ...
};
```

To export properties to SILAB:

```
Out = {
      element1.property1,
      element2.property2,
      ...
};
```

In both cases, each entry's syntax *is <element id>.<property name>*. That means: the QML **item's id** is followed by a dot, and then the **name** of the **property**.

By default, all these variables are created as floating point numbers. Optionally, string variables can be created, by appending (string) after the variable name:

```
In = {
      element1.property1,          # a numeric value
      element2.property2(string)   # a text value
};
```

For example, this can be used to change / retrieve the text of Label and TextInput elements.

7. Example:

This DPU works well using the following script:

```
DPUSGETextureQuick HMI
{
      Computer =  { LOCALHOST };
```

```
        Index = 49;

        WindowW = 1024;
        WindowH = 800;
        TextureName = "$QuickHMI";

        MainFile = "qml/minimal.qml";

        In = {
              progress.value,
              image1.opacity,
              label.text(string)
        };
        Out = {
              slider.value,
              textField.text(string)
        };
};

DPUSGEVideoObject DynObject
{
        Computer = {VIS};
        Index = 50;

        View = SGEView;
        TextureName = "$QuickHMI";

        X = 0;
        Y = 400;
        Z = 9;
        sX = 512;
        sY = 400;
};
```

The associated QML file qml/minimal.qml is:

```
import QtQuick 2.6
import QtQuick.Window 2.0
import QtQuick.Controls 1.0

Rectangle {
        id: rectangle

        width: 640
        height: 480
```

```
anchors.fill: parent

Button {
      id: button
      text: qsTr("Hallo Welt!")
      anchors.left: parent.left
      anchors.leftMargin: 10
      anchors.top: parent.top
      anchors.topMargin: 10
      opacity: 1.0
}
Button {
      id: button1
      x: 247
      y: 311
      text: qsTr("Hallo Mond!")
      visible: false
      anchors.right: parent.right
      anchors.rightMargin: 10
      anchors.bottom: parent.bottom
      anchors.bottomMargin: 10
      opacity: 0.0
}
Image {
      id: image
      width: 226
      height: 178
      anchors.left: parent.left
      anchors.leftMargin: 20
      anchors.top: button.bottom
      anchors.topMargin: 20
      fillMode: Image.PreserveAspectFit
      source: "res/biker1.png"
      opacity: 1.0
}
Image {
      id: image1
      x: 197
      y: 174
      width: 200
      height: 200
      fillMode: Image.PreserveAspectFit
      anchors.bottom: button1.top
```

```
        anchors.bottomMargin: 20
        anchors.right: parent.right
        anchors.rightMargin: 20
        source: "res/bush1.png"
        opacity: 0.0
    }
    ProgressBar {
        id: progress
        y: 447
        anchors.right: image1.left
        anchors.rightMargin: 10
        anchors.bottom: parent.bottom
        anchors.bottomMargin: 10
        anchors.left: parent.left
        anchors.leftMargin: 10
    }
    Slider {
        id: slider
        y: 412
        anchors.right: image1.left
        anchors.rightMargin: 10
        anchors.bottom: progress.top
        anchors.bottomMargin: 10
        anchors.left: parent.left
        anchors.leftMargin: 10
    }
    Connections {
        target: button
        onClicked: { rectangle.state = "Mond" }
    }
    Connections {
        target: button1
        onClicked: { rectangle.state = "" }
    }
    Connections {
        target: slider
        onValueChanged: progress.value = slider.value
    }

    Label {
        id: label
        width: 156
        height: 13
        text: qsTr("Hallo Welt!")
```

```
        font.pointSize: 18
        font.family: "Verdana"
        anchors.top: image.bottom
        anchors.topMargin: 20
        anchors.left: parent.left
        anchors.leftMargin: 20
    }
    TextField {
        id: textField
        text: "Test"
        anchors.top: label.bottom
        anchors.topMargin: 20
        anchors.left: parent.left
        anchors.leftMargin: 20
        placeholderText: qsTr("Text Field")
    }
    transitions: [
        Transition {
            from: "*"
            to: "Mond"

            NumberAnimation {
                targets: [button, image, button1, image1]
                properties: "opacity"
                duration: 1000
            }
        },
        Transition {
            from: "Mond"
            to: ""

            NumberAnimation {
                targets: [button, image, button1, image1]
                properties: "opacity"
                duration: 1000
            }
        }
    ]
    states: [
        State {
            name: "Mond"

            PropertyChanges {
            target: button
```

```
                visible: false
                opacity: 0.0
                }
                PropertyChanges {
                      target: image
                      opacity: 0.0
                }
                PropertyChanges {
                      target: button1
                      visible: true
                      opacity: 1.0
                }
                PropertyChanges {
                      target: image1
                      opacity: 1.0
                }
          }
    ]
}
```